



UNIVERSITY OF
BATH

Dual-Wavelength Pulse Oximeter

Liam Short (LCS78)

EE12001: Design Skills

University of Bath

Word count: 1762

16 January 2025

Abstract

This project involved the design and implementation of a dual-wavelength pulse oximeter using an Arduino-based embedded system. The primary objective was to measure heart rate and blood oxygen saturation (SpO₂) using photoplethysmography principles, combining optical sensing, signal conditioning, and embedded software. The system employs red and infrared light sources and a photodetector to capture pulsatile blood volume changes, which are processed to extract physiological parameters via an integrated sensor. Hardware and software integration was achieved using an Arduino microcontroller, optical sensors, and an OLED display for real-time feedback. The completed system successfully demonstrated heart rate and SpO₂ measurement under controlled conditions. This project highlights key skills in embedded system design, sensor interfacing, and real-time data processing, with relevance to medical monitoring and wearable health technologies.

Table of Contents

Abstract.....	i
Table of Contents.....	ii
List of Acronyms.....	iii
1. Introduction.....	1
1.1 Context.....	1
1.2 Project Objectives.....	1
2 System Design.....	1
2.1 Hardware Design.....	1
2.2 Software Design.....	1
3 Results.....	2
4 Discussion.....	2
5 Conclusion.....	2
References.....	3
Appendices.....	4
Appendix A: Arduino code.....	4

List of Acronyms

UoB	University of Bath
PPG	Photoplethysmography
SpO2	Peripheral Capillary Oxygen Saturation
OLED	Organic Light Emitting Diode
PCB	Printed Circuit Board
I2C	Inter-Integrated Circuit

1.Introduction

1.1 Context

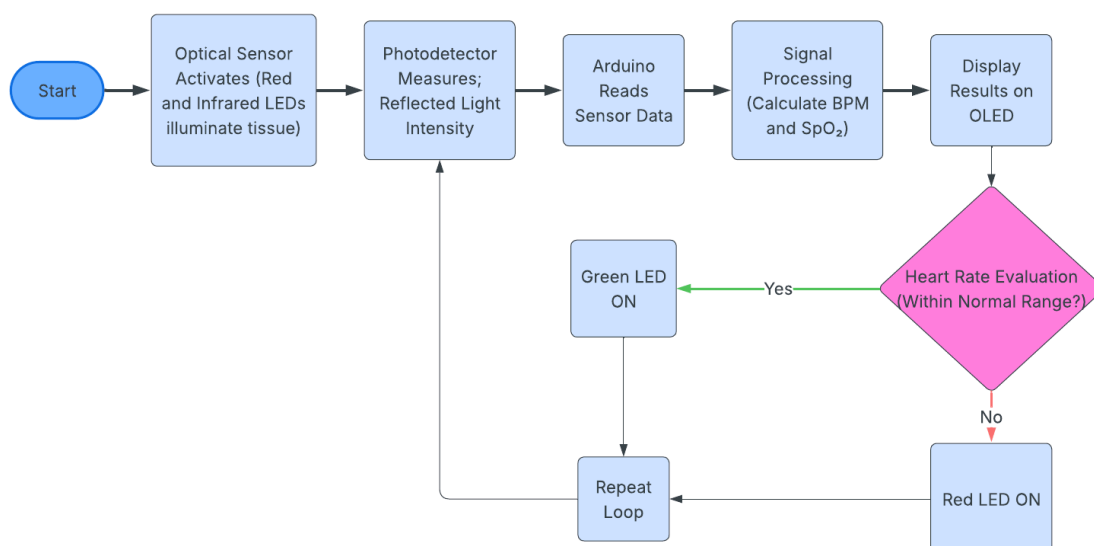
Pulse oximetry is a non-invasive technique widely used in medical and fitness devices to monitor heart rate and blood oxygen saturation. Dual-wavelength pulse oximeters use red and infrared light to distinguish between oxygenated and deoxygenated haemoglobin, enabling SpO₂ estimation. Embedded systems such as the Arduino Nano provide a platform for implementing these sensing techniques.

1.2 Project Objectives

- Interface a dual-wavelength optical sensing system with an Arduino Nano
- Implement real-time data acquisition and processing
- Display physiological data clearly using an OLED interface

2 System Design

The system integrates a MAX30100 and embedded software to form a compact dual-wavelength pulse oximeter capable of measuring heart rate and blood oxygen saturation (SpO₂). An Arduino Nano acts as the central controller, coordinating sensor data acquisition, signal processing, and user interface output. Red and infrared light sources illuminate tissue, and a photodetector captures variations in light absorption caused by pulsatile blood flow. These signals are processed in real time to extract physiological parameters, which are displayed on an OLED screen. User input is provided through push buttons to allow switching between display modes, while status LEDs give immediate visual feedback on heart rate range.

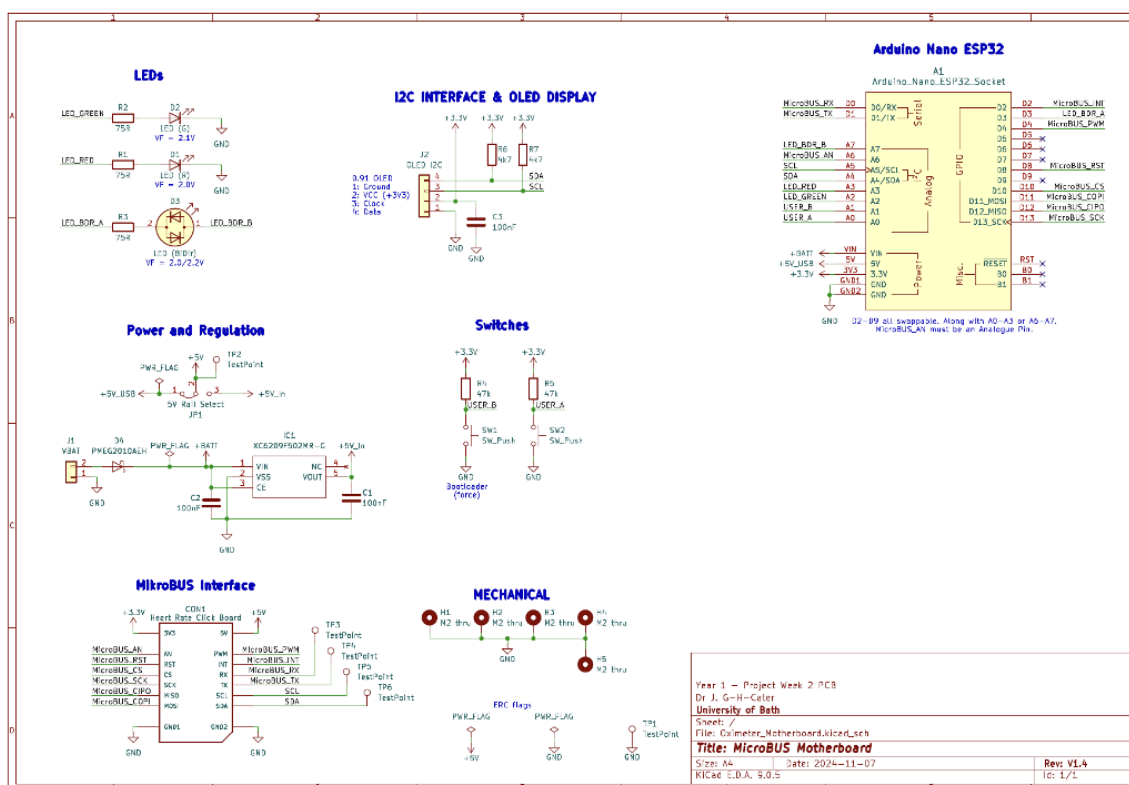


System Operation Flow

2.1 Hardware Design

The hardware design is organised into clearly defined functional blocks, including the Arduino Nano ESP32 controller, optical sensing module, OLED display, user interface components, and power regulation circuitry, as shown in the system schematic. The MAX30100 pulse oximeter sensor is interfaced via the MikroBUS connector, allowing red and infrared photoplethysmography signals to be acquired using a standardised module. The MAX30100 sensor internally manages the red and infrared LED sequencing and photodetector signal acquisition, allowing the Arduino to access processed heart rate and SpO2 values directly via the sensor library. Both the sensor and OLED display communicate with the microcontroller over a shared I2C bus, reducing pin usage while maintaining reliable data transfer through the use of appropriate pull-up resistors.

Status LEDs are driven directly from GPIO pins via current-limiting resistors to provide immediate visual feedback on system operation and heart rate range. Power is supplied via USB or battery input and regulated to appropriate voltage levels, with decoupling capacitors included to improve supply stability and reduce noise in the analog sensing circuitry. Modular connectors and pin headers are used throughout the design, enabling component reuse and improving repairability at the end of the product's life.



PCB Wiring Diagram

2.2 Software Design

The software is written using the Arduino programming framework and structured around the standard `setup()` and `loop()` functions. During initialisation, communication with the sensor and display is established, and input/output pins are configured. Sensor data is updated continuously, while heart rate and SpO2 values are calculated at fixed time intervals using non-blocking timing based on the `millis()` function.

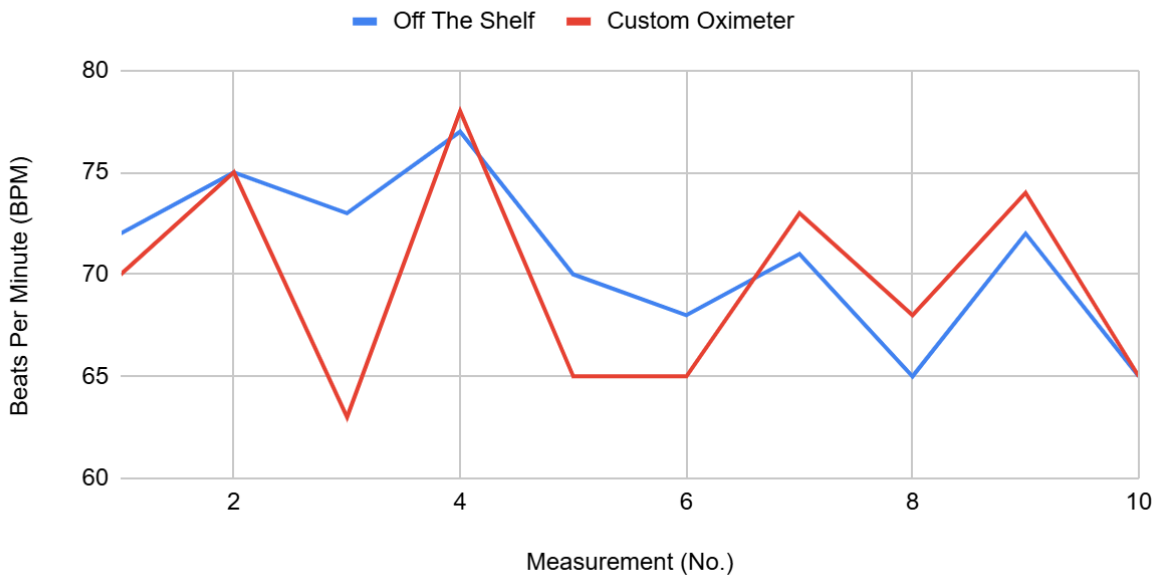
2.3

A menu system implemented using conditional logic allows users to switch between different display modes. Software debouncing is applied to button inputs to ensure reliable operation. The code is divided into modular functions for sensor updates, display handling, and LED control, improving readability and maintainability while avoiding the need for physical circuit changes.

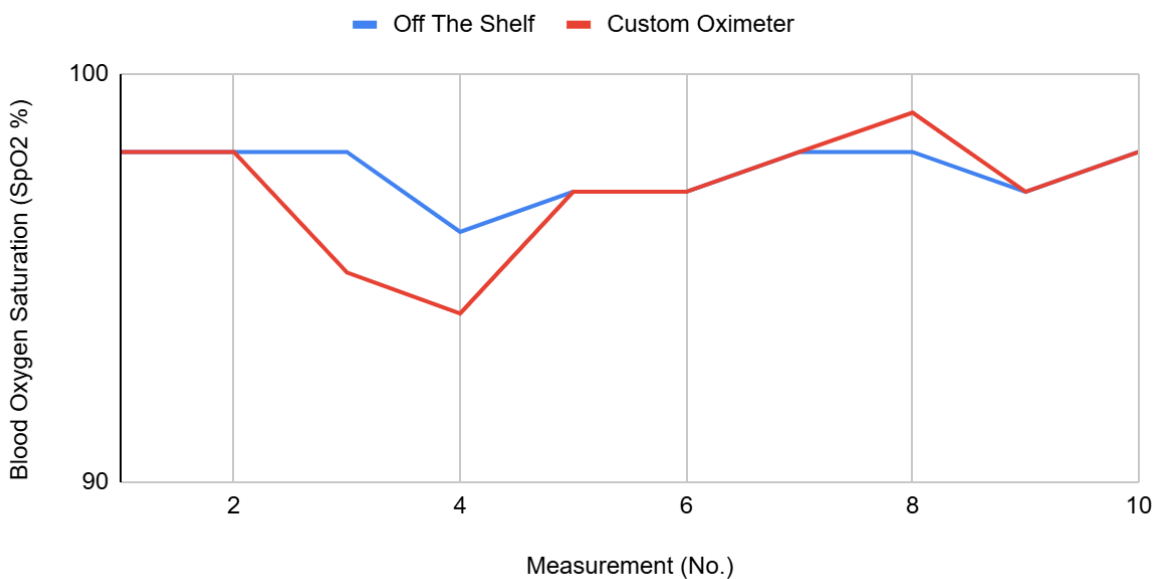
3 Results

Comparison of BPM and SpO2 Measurements Between a Commercial Pulse Oximeter and This Custom Dual-Wavelength Oximeter				
	Commercial		Custom Oximeter	
Measurement (No.)	BPM	SpO2	BPM	SpO2
1	72	98	70	98
2	75	98	75	98
3	73	98	63	95
4	77	96	78	94
5	70	97	65	97
6	68	97	65	97
7	71	98	73	98
8	65	98	68	99
9	72	97	74	97
10	65	98	65	98

Heart Rate (BPM) Comparison Between Commercial and Custom Oximeter



Blood Oxygen Saturation (SpO₂) Comparison Between Commercial and Custom Oximeter



3.1 Description and Analysis of Results

The results compare heart rate (BPM) and blood oxygen saturation (SpO₂) measurements obtained from a commercial pulse oximeter and the custom dual-wavelength oximeter across ten consecutive readings.. Overall, the BPM values measured by the custom system closely follow those recorded by the commercial device, with most readings differing by only a few beats per minute. This indicates

that the custom oximeter is able to detect pulsatile blood flow reliably and produce heart rate values consistent with a clinically available reference device.

SpO₂ measurements from the custom oximeter also show good agreement with the commercial oximeter, with values generally within 1–3% across all measurements. Slightly lower SpO₂ readings are observed in some cases, which may be attributed to factors such as sensor placement, ambient light interference, or limitations in signal processing and calibration. Occasional larger BPM deviations, such as in measurement three, suggest sensitivity to motion or noise in the photoplethysmography signal.

3.2 Conclusions

The results demonstrate that the custom dual-wavelength pulse oximeter is capable of producing heart rate and SpO₂ measurements that are broadly comparable to those of a commercial device. While minor discrepancies exist, the overall trends and ranges are consistent, indicating that the system functions correctly. Further improvements, such as enhanced filtering and calibration, could reduce measurement variation and improve accuracy.

4 Discussion

Several aspects of the project worked well, particularly the integration of the optical sensor, microcontroller, and OLED display into a cohesive embedded system. The use of a shared I²C bus allowed efficient communication between the sensor and display, while modular hardware connections improved reliability and ease of debugging. The software structure, based on non-blocking timing and modular functions, enabled stable real-time updates without interrupting sensor data acquisition.

Challenges were encountered in achieving consistent and stable measurements, as photoplethysmography signals are sensitive to noise, finger placement, and motion. Variations in ambient light and small movements occasionally caused fluctuations in BPM and SpO₂ readings.

A potential improvement would be implementing additional digital filtering to reduce signal noise, refining calibration to improve SpO₂ accuracy. Designing a dedicated enclosure for the system that shields the optical sensor from ambient light would help reduce measurement noise and improve the stability and accuracy of BPM and SpO₂ readings.

5 Conclusion

Successful implementation of an integrated dual-wavelength pulse oximeter system capable of measuring heart rate and blood oxygen saturation.

Practical understanding of Arduino programming, optical sensor interfacing, and real-time data acquisition.
Recognition of the importance of signal processing and reliable sensor feedback in real-world biomedical monitoring applications.

Appendices

Appendix A: Arduino code

```
/*
Co-Created by:
Liam Short (LCS78)
Luke Averill (LA2005)
Joseph Johnson (JJ2269)

Project:
Dual-Wavelength Pulse Oximeter with OLED Display and Status LEDs
*/

// =====
// Library Includes
// =====
#include "MAX30100_PulseOximeter.h" // MAX30100 sensor library
#include <SPI.h>                     // SPI library (required by some
displays)
#include <Wire.h>                    // I2C communication library
#include <Adafruit_GFX.h>            // Graphics library
#include <Adafruit_SSD1306.h>        // OLED display library

// =====
// Constant Definitions
// =====
#define REPORTING_PERIOD_MS 1000    // Screen update interval (1
second)
#define SCREEN_WIDTH 96             // OLED width in pixels
#define SCREEN_HEIGHT 16            // OLED height in pixels
#define OLED_RESET -1               // OLED reset pin (not used)
#define SCREEN_ADDRESS 0x3C         // I2C address of OLED

// =====
// Object Declarations
// =====
```

```

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);
PulseOximeter pox;

// =====
// Global Variables
// =====
int menu = 0; // Display menu state (0-2)
int heartRate = 0; // Measured heart rate (BPM)
int SpO2 = 0; // Measured blood oxygen saturation
(%)
uint32_t tsLastReport = 0; // Timestamp of last display
update

// =====
// I/O Pin Definitions
// =====
int switchA = A0; // Menu decrement button
int switchB = A1; // Menu increment button
int ledGreen = A3; // Green status LED
int ledRed = A2; // Red status LED
int ledBdrA = D3; // Border LED A
int ledBdrB = A7; // Border LED B

// =====
// Debounce Variables
// =====
int buttonStateA;
int buttonStateB;
int lastButtonStateA = HIGH;
int lastButtonStateB = HIGH;
unsigned long lastDebounceTimeA = 0;
unsigned long lastDebounceTimeB = 0;
unsigned long debounceDelay = 100; // Debounce delay in ms

// =====
// Setup Function
// =====
void setup()
{
    Serial.begin(9600);
    Serial.println("Initializing pulse oximeter...");

```

```

// Initialise MAX30100 sensor
if (!pox.begin()) {
    Serial.println("MAX30100 init failed");
    while (1); // Halt if sensor fails
}

// Initialise OLED display
if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    while (1); // Halt if display fails
}

// Configure display settings
display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(0, 0);
display.cp437(true);
display.display();

// Configure I/O pin modes
pinMode(switchA, INPUT);
pinMode(switchB, INPUT);
pinMode(ledGreen, OUTPUT);
pinMode(ledRed, OUTPUT);
pinMode(ledBdrA, OUTPUT);
pinMode(ledBdrB, OUTPUT);
}

// =====
// Main Loop
// =====
void loop()
{
    pox.update(); // Update sensor readings
    updateLEDS(); // Update LEDs based on heart rate

    // Update display at fixed interval
    if (millis() - tsLastReport > REPORTING_PERIOD_MS) {
        heartRate = pox.getHeartRate();
        SpO2 = pox.getSpO2();
        writeToScreen();
        tsLastReport = millis();
    }
}

```

```

// Read button states
int readingA = digitalRead(switchA);
int readingB = digitalRead(switchB);

// Reset debounce timers if state changes
if (readingA != lastButtonStateA) lastDebounceTimeA = millis();
if (readingB != lastButtonStateB) lastDebounceTimeB = millis();

// Handle button A (menu down)
if ((millis() - lastDebounceTimeA) > debounceDelay) {
    if (readingA != buttonStateA) {
        buttonStateA = readingA;
        if (buttonStateA == HIGH) menu--;
    }
}

// Handle button B (menu up)
if ((millis() - lastDebounceTimeB) > debounceDelay) {
    if (readingB != buttonStateB) {
        buttonStateB = readingB;
        if (buttonStateB == HIGH) menu++;
    }
}

lastButtonStateA = readingA;
lastButtonStateB = readingB;

// Constrain menu range
if (menu > 2) menu = 0;
if (menu < 0) menu = 2;
}

// =====
// Display Output Function
// =====
void writeToScreen()
{
    display.clearDisplay();

    switch (menu)
    {
        case 0: // Heart rate and SpO2

```

```

        display.setTextSize(1);
        display.setCursor(0, 0);
        display.print("HR: ");
        display.print(heartRate);
        display.println(" bpm");
        display.setCursor(0, 8);
        display.print("SpO2: ");
        display.print(SpO2);
        display.println(" %");
        break;

    case 1:                                // Heart rate only
        display.setTextSize(2);
        display.setCursor(0, 0);
        display.print("HR:");
        display.print(heartRate);
        display.println("bpm");
        break;

    case 2:                                // SpO2 only
        display.setTextSize(2);
        display.setCursor(0, 0);
        display.print("SpO2:");
        display.print(SpO2);
        display.println("%");
        break;
}

display.display();
}

// =====
// LED Status Function
// =====
void updateLEDS()
{
    // Outside normal heart rate range
    if (heartRate > 110 || heartRate < 60)
    {
        digitalWrite(ledRed, HIGH);
        digitalWrite(ledGreen, LOW);
        digitalWrite(ledBdrA, HIGH);
        digitalWrite(ledBdrB, LOW);
    }
}

```

```
}  
else  
{  
    digitalWrite(ledRed, LOW);  
    digitalWrite(ledGreen, HIGH);  
    digitalWrite(ledBdrA, LOW);  
    digitalWrite(ledBdrB, HIGH);  
}  
}
```